

Driver Fatigue Detection with Gesture-Based Activation

Blanca Valdés, Felix Westin, George Vashakidze, Joud Taher, Kenny Tohme, William Yammine
Bachelor in Computer Science and Artificial Intelligence

5 May 2026

Abstract—Driver fatigue is a relevant safety problem because reduced attention, long eye closure, yawning, and unstable head posture can indicate that a driver is becoming less alert. In this project, we designed and implemented a computer vision system that detects simulated driver fatigue inside a stationary vehicle. The system remains inactive by default and only starts monitoring after a predefined gesture sequence is performed in front of the camera. After activation, it analyzes facial landmarks and face-crop sequences using both classical computer vision methods and a modern CNN-LSTM model. The classical machine learning model achieved an accuracy of 0.787 and a drowsy-class F1 score of 0.811. The modern CNN-LSTM model achieved an accuracy of 0.819 and a drowsy-class F1 score of 0.794 on the validation split. These results show that the system can detect several important fatigue cues in a realistic parked-car setup, although further testing with more subjects and lighting conditions would be needed for stronger generalization.

Index Terms—driver fatigue detection, computer vision, gesture activation, facial landmarks, EAR, MAR, CNN-LSTM

I. INTRODUCTION

Fatigue detection is a common application of computer vision because many signs of tiredness are visible in the face and head. For example, a tired driver may keep their eyes closed for longer than normal, yawn repeatedly, nod their head downwards, or show unstable head posture. A camera-based system can monitor these cues without requiring the driver to wear extra sensors.

The goal of this project was to build a driver fatigue detection system for a bachelor-level computer vision course. A central requirement was that the system must not begin monitoring immediately. Instead, the system must remain inactive until a specific gesture sequence is performed correctly. This makes the project different from a standard fatigue detector, because it combines hand gesture recognition, state-machine logic, facial feature extraction, fatigue classification, and real-time user feedback.

The final system was designed for use inside a parked vehicle. No real driving was performed during data collection or testing. This was important both for safety and for matching the assignment constraints.

II. SYSTEM OVERVIEW

The system uses a camera positioned like an in-car monitoring camera. The user sits in the driver's seat, and the application processes the camera stream in real time. The pipeline has four main stages:

1) gesture recognition using MediaPipe Hands,

- 2) activation control using a finite-state machine,
- 3) fatigue detection using classical features and a CNN-LSTM model,
- 4) fusion, alerting, and visualization through an OpenCV overlay.

The main runtime script is `main.py`. When the program starts, it opens the camera and displays the interface, but fatigue detection is not active yet. Only after the activation gesture sequence is completed does the system begin analyzing the driver's face.

III. GESTURE-BASED ACTIVATION

The required activation sequence is:

`OPEN_PALM → THUMBS_UP`.

The two gestures must be recognized in the correct order and completed within 5 seconds. If the user performs the wrong order, waits too long, or does not complete the sequence, the system remains inactive. The sequence and time window are configured in `config.py`.

The gesture detector is implemented in `gesture/detector.py` using MediaPipe Hands [1]. It classifies a visible hand as either `OPEN_PALM`, `THUMBS_UP`, or no recognized gesture. The activation logic is separated into `gesture/state_machine.py`. This separation made the system easier to test because the gesture sequence can be validated without needing a live camera.

The state machine also supports deactivation using the same gesture sequence while the system is already active. This is not strictly required by the assignment, but it makes the demo and user interaction cleaner.

IV. FATIGUE DETECTION METHODS

A. Facial Landmark Extraction

The classical part of the system uses MediaPipe Face Mesh [1] to extract facial landmarks. These landmarks are converted into pixel coordinates and then used to compute visual fatigue features. The landmark extraction code is located in `classical/landmarks.py`.

B. Classical Threshold-Based Detection

The first fatigue detector is based on hand-crafted features and thresholds. This approach is interpretable because each alert is connected to a specific visual cue. The main features are:

- Eye Aspect Ratio (EAR), used to detect long eye closure.
- Mouth Aspect Ratio (MAR), used to detect yawning.
- Head pitch and roll, estimated using solvePnP.
- Blink-rate estimate.

The threshold detector is implemented in `classical/detector.py`. It raises alerts for long eye closure, yawning, head nodding, and head tilt. Head pose is calibrated during the first frames after activation so that the system compares later head movement against the user's normal seated posture.

C. Classical Machine Learning

In addition to fixed thresholds, we trained a classical machine learning classifier using the feature vector:

[EAR, MAR, pitch, yaw, roll, blink rate].

The classifier combines an SVM with an RBF kernel and a Random Forest using soft voting. This gives the model a learned decision boundary while still using interpretable features. The training code is in `classical/train.py`, and the trained model is saved as `models/classical_classifier.joblib`.

D. Modern CNN-LSTM Model

The modern model is a CNN-LSTM architecture. A MobileNetV2 backbone [2] extracts visual features from face crops, and a two-layer LSTM models temporal information over a sequence of 30 frames. The output is a binary prediction: alert or drowsy.

The model architecture is implemented in `modern/model.py`, and the training script is `modern/train.py`. For the final run, the MobileNetV2 backbone was initialized with pretrained weights and frozen. Its features were precomputed, which made it practical to train the LSTM on CPU. The trained model is saved as `models/fatigue_net.pth`.

E. Fusion

The fusion module combines the classical fatigue score and the CNN-LSTM score. If the modern model is missing, the system falls back to the classical score only. If both models are available, the final fatigue confidence is computed using weighted fusion. This is implemented in `fusion/combiner.py`.

V. DATASET AND EXPERIMENTAL SETUP

All final images were captured inside a stationary vehicle with the subject seated in the driver's position. The camera position was chosen to mimic an in-car monitoring setup. The dataset includes both fatigue-related frames and gesture frames.

The fatigue dataset contains 1635 alert frames, 852 eyes-closed frames, 464 yawning frames, and 1172 head-down frames. For binary classification, the fatigue labels were grouped into 1635 alert frames and 2488 drowsy frames.

The gesture dataset contains examples of open palm, thumbs up, no gesture, correct sequence, and wrong sequence. The final evaluation used only the group's captured in-car data.

The classical classifier was trained from MediaPipe landmark features extracted from the fatigue frames. During feature extraction, 3920 usable rows were obtained, while 203 frames were skipped because no usable face input was detected.

The modern CNN-LSTM model was trained using 867 training sequences and 144 validation sequences. Each sequence contained 30 frames, and the sliding-window stride was 3 frames. The model was trained for 15 epochs, with the best checkpoint selected using validation F1 score.

VI. RESULTS

A. Classical Machine Learning Results

The classical ML model was trained on 3136 samples and tested on 784 samples. Table I summarizes the main results.

TABLE I
VALIDATION AND TEST RESULTS FOR THE TWO TRAINED FATIGUE CLASSIFIERS.

Metric	Classical ML	CNN-LSTM
Accuracy	0.787	0.819
Precision (drowsy)	0.849	0.658
Recall (drowsy)	0.777	1.000
F1 (drowsy)	0.811	0.794
ROC-AUC	0.894	–

The relatively high precision of the classical model means that when it predicts drowsiness, it is often correct. Its recall is lower than the modern model, meaning it misses some drowsy cases. This is a reasonable behavior for a feature-based classifier, especially because fatigue cues can vary between yawning, eye closure, and head pose.

B. Modern CNN-LSTM Results

The modern model achieved an accuracy of 0.819 and a drowsy-class F1 score of 0.794. The recall of 1.000 means that the model detected all drowsy validation sequences. However, the lower precision shows that it also classified some alert sequences as drowsy. In a fatigue detection system this tradeoff can be acceptable, because missing a fatigued driver is usually more serious than producing an occasional false alarm. Still, this result also suggests that the model is conservative and should be tested on more subjects before being considered robust.

C. Real-Time System Behavior

In the live application, the system starts inactive and displays the activation instructions. After the correct OPEN_PALM to THUMBS_UP sequence is performed, the interface changes to active monitoring. The overlay displays current fatigue metrics such as EAR, MAR, pitch, roll, and the final fatigue score. When fatigue is detected, an alert label is shown and an audio alert is played.

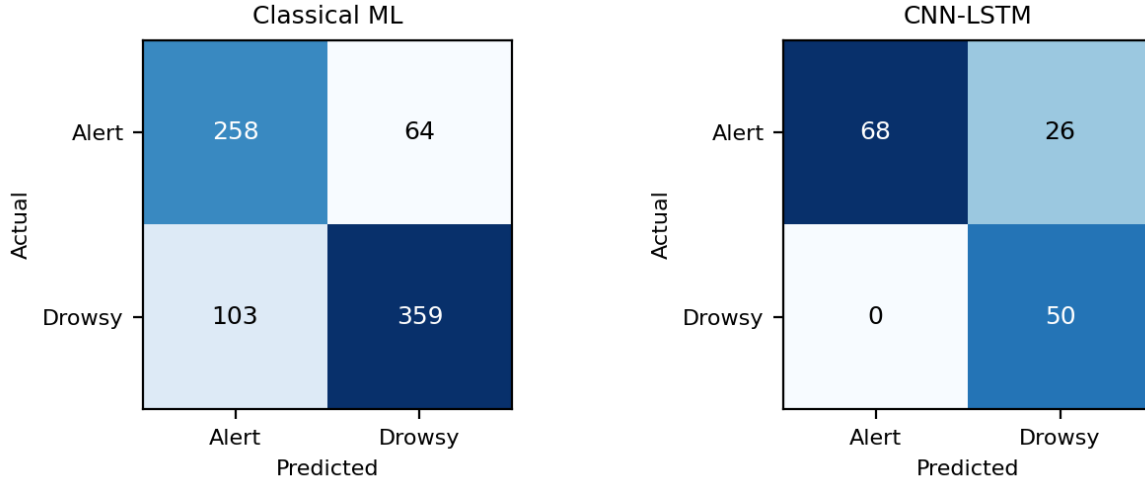


Fig. 1. Confusion matrices for the classical ML classifier and the CNN-LSTM model. The CNN-LSTM detects all drowsy validation sequences, but it also produces more false drowsy predictions on alert sequences.

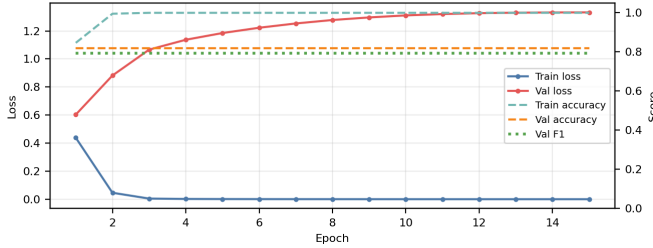


Fig. 2. CNN-LSTM training history. The training loss quickly decreases, while validation loss increases after the first epoch, suggesting overfitting on the current dataset.

VII. DISCUSSION

The project combines a classical and a modern approach, which is useful because the two methods have different strengths. The threshold-based detector is easy to understand and gives direct explanations for alerts. For example, if the MAR value is high for several frames, the system can clearly report a yawn. This makes debugging easier and is helpful for a course project because the visual reasoning is visible.

The classical ML classifier improves on the fixed threshold idea by learning from the collected data. It still uses understandable features, but it can combine them in a more flexible way than manually chosen rules. Its results were strong enough to support using it in the final fusion pipeline.

The CNN-LSTM model achieved better accuracy than the classical ML model and had perfect drowsy recall on the validation split. At the same time, the training history shows signs of overfitting after the first epoch, because the training accuracy quickly reached 1.000 while validation loss increased. This is not surprising because the dataset is relatively small and likely contains similar frames from the same recording conditions. The result is still useful for demonstrating a modern pipeline, but it should not be interpreted as proof that

the model would generalize to all drivers and vehicles.

Overall, the fusion design is a good practical compromise. The classical detector provides stable and interpretable behavior, while the modern model adds temporal learning from face appearance. The system also remains usable if one model is unavailable, because it can fall back to the classical threshold score.

VIII. LIMITATIONS

The main limitation is dataset size and diversity. The current dataset is enough for a working prototype, but a real fatigue detection system would need more subjects, different lighting conditions, different camera positions, glasses/no-glasses cases, and more natural fatigue examples.

Another limitation is that the fatigue examples are simulated. Simulated yawning, eye closure, and head nodding are useful for a safe classroom experiment, but they are not exactly the same as real drowsiness during long driving. Also, the validation split is based on collected frame sequences, so there may still be visual similarity between training and validation data.

Finally, the system is designed only for use in a parked vehicle, following the assignment rules. It should not be tested during real driving.

ETHICAL IMPACT STATEMENT

The system is intended only as a prototype for a computer vision course and should not be used as a real driver safety product. A fatigue detection system can affect user privacy because it records a person's face inside a vehicle. For this reason, any real deployment would need clear consent, local data processing when possible, secure storage, and transparent information about what is recorded. There is also a safety risk if users trust the system too much. The system should be considered an assistive warning tool, not a replacement for responsible driving or rest.

IX. CONCLUSION

This project implemented a complete computer vision system for driver fatigue detection with gesture-based activation. The system remains inactive until the user performs the correct two-gesture sequence within the time limit. After activation, it detects fatigue cues using facial landmarks, classical thresholds, a trained classical ML classifier, and a CNN-LSTM model.

The classical ML model achieved a drowsy F1 score of 0.811, while the modern CNN-LSTM model achieved a drowsy F1 score of 0.794 and detected all drowsy validation sequences. These results show that the system satisfies the main project requirements and demonstrates both traditional and modern computer vision approaches. Future work should focus on collecting more diverse data and testing the system under more varied in-car conditions.

AUTHOR CONTRIBUTIONS

Felix Westin implemented the initial system architecture, including the real-time pipeline, gesture activation logic, classical fatigue detection modules, model scaffolding, fusion logic, UI overlay, and unit tests. Joud Taher collected and organized the in-car image dataset used for gesture recognition and fatigue detection experiments. Blanca Valdés completed the final project integration by fixing the training pipeline, training and evaluating the classical and modern models, generating result plots, preparing the README, and writing the final ACII-style report. George Vashakidze tested the live camera-based detection system and helped prepare the final demonstration video, which Felix filmed, showing gesture activation and simulated fatigue detection. Kenny Tohme helped review the assignment requirements and contributed to defining the gesture-activation sequence used in the final system. William Yammine helped review the dataset categories and provided feedback on the final demonstration flow.

REFERENCES

- [1] C. Lugaresi et al., “MediaPipe: A framework for building perception pipelines,” arXiv preprint arXiv:1906.08172, 2019.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.
- [3] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.